

ARQUITETURA ZION

PADRÕES PARA LIBERTAR SEU CÓDIGO



Aprenda de forma simples e objetiva os principais padrões de design usados no desenvolvimento moderno.

BRUNO EMIDIO



PADRÕES DE DESIGN

Design patterns são soluções reutilizáveis para problemas recorrentes no desenvolvimento de software. Eles não são códigos prontos, mas estruturas conceituais que ajudam a tornar o sistema mais organizado, flexível e fácil de manter. Utilizar padrões acelera o desenvolvimento, reduz erros e facilita a comunicação entre desenvolvedores, porque todos passam a trabalhar com um vocabulário comum.

01

PADRÕES CRIACIONAIS

Os padrões criacionais lidam com como objetos são criados, ajudando a tornar o código mais flexível, modular e desacoplado



SINGLETON

O padrão Singleton garante que uma classe tenha apenas uma instância durante toda a execução da aplicação e fornece um ponto global de acesso a ela. Ele é útil quando um recurso deve ser compartilhado, como configurações globais ou conexões controladas.

```
Singleton.java

public class ConexaoBD {
    private static ConexaoBD instancia;

    private ConexaoBD() {}

    public static ConexaoBD getInstancia() {
        if (instancia == null) {
            instancia = new ConexaoBD();
        }
        return instancia;
    }
}
```



FACTORY METHOD

O Factory Method delega a criação de objetos para subclasses, permitindo que cada tipo concreto defina sua própria forma de criação. É usado quando o código precisa trabalhar com objetos sem conhecer seus tipos exatos.

```
Factory.java

abstract class Transporte {
    abstract Veiculo criarVeiculo();
}

class TransporteTerrestre extends Transporte {
    Veiculo criarVeiculo() {
        return new Carro();
    }
}
```



ABSTRACT FACTORY

O Abstract Factory cria famílias de objetos relacionados sem expor suas implementações concretas. É ideal para sistemas que precisam trocar facilmente de contexto ou tema.

```
AbstractFactory.java

interface UIFactory {
    Botao criarBotao();
    Janela criarJanela();
}

class DarkUIFactory implements UIFactory {
    public Botao criarBotao() {
        return new BotaoDark();
    }
    public Janela criarJanela() {
        return new JanelaDark();
    }
}
```

02

PADRÕES ESTRUTURAIS

Os padrões estruturais explicam como classes e objetos podem ser combinados para formar estruturas mais eficientes e flexíveis.



ADAPTER

0 Adapter converte a interface de um objeto em outra esperada pelo cliente, permitindo integrar classes incompatíveis sem modificar seu código.

```
Adapter.java

interface TomadaBR {
    void conectar();
}

class TomadaEU {
    void plugUE() {
        System.out.println("Conectado padrão europeu");
    }
}

class AdapterEUparaBR implements TomadaBR {
    private TomadaEU tomada;

    AdapterEUparaBR(TomadaEU tomada) {
        this.tomada = tomada;
    }

    public void conectar() {
        tomada.plugUE();
    }
}
```



DECORATOR

O Decorator adiciona comportamentos dinamicamente a objetos sem alterar sua classe base, útil para extensões flexíveis.

```
Decorator.java

interface Cafe {
    double preco();
}

class CafeSimples implements Cafe {
    public double preco() {
        return 5.0;
    }
}

class ComLeite implements Cafe {
    private Cafe base;
    ComLeite(Cafe base) {
        this.base = base;
    }

    public double preco() {
        return base.preco() + 2.0;
    }
}
```



FACADE

O Facade fornece uma interface simplificada para um conjunto complexo de funcionalidades internas, ideal para reduzir acoplamento.

```
Facade.java

class SistemaFacade {
    private Audio audio = new Audio();
    private Video video = new Video();

    public void iniciar() {
        audio.iniciar();
        video.iniciar();
    }
}
```

03

PADRÕES COMPORTAMENTAIS

Os padrões comportamentais focam em como objetos interagem, se comunicam e distribuem responsabilidades.



OBSERVER

O Observer notifica automaticamente múltiplos objetos interessados quando ocorre uma mudança no objeto observado. Ele funciona registrando observadores e emitindo notificações.

```
Observer.java

interface Observador {
    void atualizar(String msg);
}

class Sujeito {
    private List<Observador> obs = new ArrayList<>();
    public void adicionar(Observador o) {
        obs.add(o);
    }
    public void notificar(String msg) {
        obs.forEach(o -> o.atualizar(msg));
    }
}
```



STRATEGY

O Strategy permite trocar algoritmos dinamicamente ao extrair o comportamento para classes separadas. Ele funciona recebendo uma estratégia no momento da execução.

```
Strategy.java

interface Frete {
    double calcular(double peso);
}

class FreteExpresso implements Frete {
    public double calcular(double peso) {
        return peso * 10;
    }
}

class Entrega {
    private Frete frete;
    public Entrega(Frete frete) {
        this.frete = frete;
    }
    public double calcular(double peso) {
        return frete.calcular(peso);
    }
}
```



COMMAND

O Command encapsula uma ação como um objeto, permitindo desfazer operações e criar filas de comandos reutilizáveis.

```
Command.java

interface Comando {
    void executar();
}

class Luz {
    void ligar() {
        System.out.println("Luz ligada.");
    }
}

class LigarLuz implements Comando {
    private Luz luz;

    public LigarLuz(Luz l) {
        this.luz = l;
    }

    public void executar() {
        luz.ligar();
    }
}
```

AGRADECIMENTOS



OBRIGADO PELA LEITURA!

O conteúdo do e-book foi gerado por IA, revisado e então transformado nesse formato por um humano.

Agradeço a Universia e a DIO por fornecerem o material de apoio e guias que foram utilizados como base para produção deste e-book.

Os arquivos e prompts utilizados podem ser encontrados em:



<https://github.com/B-Emidio/ebook-ia-dio>